Travaux pratiques n°2	#Réseaux de neurones convolutifs
Matière : Deep Learning	**Enseignants :** Taoufik Ben Abdallah
Discipline : 2ème année Génie Informatique	Tarak Ben Said
Année Universitaire : 2026-2027 / S1	

L'objectif de ce TP vise à construire un modèle de réseau de neurones convolutifs pour détecter si une personne porte une bavette (**`mask`**) ou **non** en utilisant la base de données **`Face-Mask`**. Cette dernière représente un ensemble d'images de visage en couleur, de taille variée, qui sont annotées par deux classes : **`"WithMask"`** ou **`"WithoutMask"`**. Elle est composée d'un **ensemble d'apprentissage** de **`4241`** images (**`2087 "WithMask"`** et **`2154 "WithoutMask"`**), d'un **ensemble de validation** de **`1053`** images (**`522 "WithMask"`** et **`531 "WithoutMask"`**), et d'un **ensemble de test** de **`2259`** images (**`1116 "WithMask"`** et **`1143 "WithoutMask"`**).

NB.

- Créer une copie du fichier `.zip` de la base de données **`Face-Mask`** (**`FaceMask.zip`**) dans votre drive à partir du lien :
  https://drive.google.com/file/d/1BQIE9RQsrGN6ZyRCPZ_1f1Zz-1YpLutH/view?usp=drive_link
- Lancer Google Colab et téléchargez le fichier **`TP2_DL.ipynb`**. Ensuite, exécuter la première cellule du notebook qui vise à l'importation des bibliothèques, la deuxième cellule qui tente à l'assemblage de votre compte **Google Drive** dans **Colaboratory** et la troisième cellule qui décompresse le fichier FaceMask.zip dans le répertoire **Face Mask Dataset**

Travail à faire :

1/ Charger les données d'apprentissage, de validation et de test de la base de données **`Face-Mask`** respectivement dans des conteneurs de type **`BatchDataset`** **`train_ds`**, **`val_ds`**, et **`test_ds`**. Spécifier la taille des images à **`28×28`** pixels et la taille de batch à **`64`** (utiliser **`image_dataset_from_directory`** de **`tf.keras.utils`**)

NB. **`PrefetchDataset`** est un objet contenant des lots (batches) de taille fixe dont chaque lot contient **`batch_size`** images. Il comporte :

- Un **`EagerTensor`** (matrice à **`4`** dimensions) de la forme (**`batch_size`**, **`H_image`**, **`L_image`**, **`nb_channels`**) qui représente les images de chaque batch
- Un **`EagerTensor`** `1D` de la forme (batch_size,) qui représente les labels des images de chaque batch

2/ Afficher la taille du premier batch de **`train_ds`**. Puis, afficher les labels (classes) du dataset

3/ Afficher les **`9`** premières images de l'ensemble d'apprentissage du **premier batch** (**`3`** images dans chaque ligne) (**Spécifier le nom de la classe sous chaque image**)

4/ Apprentissage d'un réseau de neurones convolutif Fmask_cnn

Construire un modèle, nommé **Fmask_cnn**, de type **`Sequential`** en respectant l'architecture présentée dans la **Figure 1**

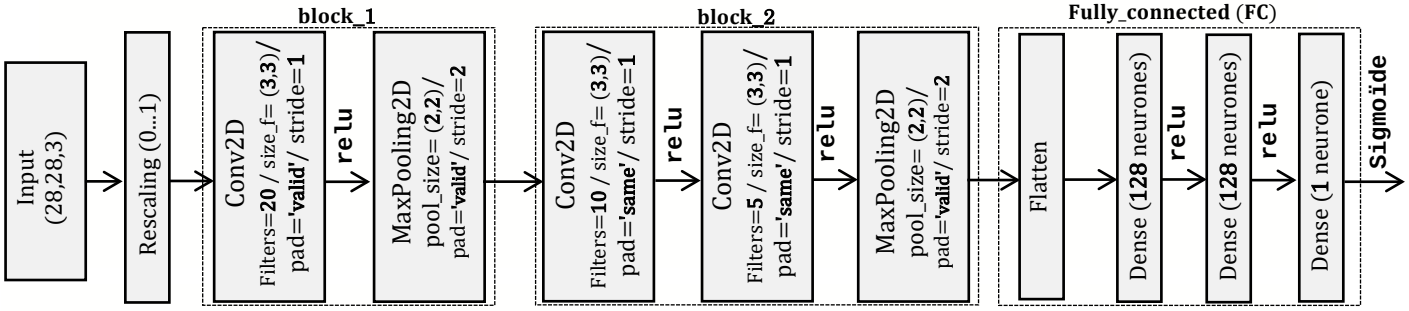


***Figure1 : Architecture CNN de* Fmask_cnn**

5/ Afficher le résumé de l'architecture de **Fmask_cnn**. Puis, représenter graphiquement l'architecture de **Fmask_cnn (**utiliser **`plot_model`** de **`tf.keras.utils`**).

6/ Compiler et exécuter **Fmask_cnn** en spécifiant l'optimiseur à **`adam`**, le *nombre d'époques* à **`10`** et la *taille de batch* égal à **`64`**

NB. Afin d'engendrer un gain de temps d'exécution sur le traitement de données, il suffit de forcer le maintien des données en cache dans la mémoire pour éviter la recharge des images à chaque fois (**`BatchDataset.cache()`**) ainsi qu'activer le traitement parallèle des batches (**`BatchDataset.prefetch()`**)

```
train_ds = train_ds.cache().prefetch(buffer_size= tf.data.AUTOTUNE)
# ------
```

- **`cache()`**: Stocke les données chargées en mémoire, ce qui évite de les recharger à chaque itération.
- **`prefetch()`**: Précharge les données en arrière-plan pendant que le modèle s'entraîne, ce qui permet de superposer les opérations d'entrée/sortie et de calcul.
- **`buffer_size`** : Détermine la quantité de données à stocker en mémoire après le chargement (**`tf.data.AUTOTUNE`**: le nombre d'éléments optimal que le buffer peut stocker)

Enregistrer le meilleur modèle en termes de `val_accuracy` dans votre Google Drive

7/ Représenter la courbe d'évaluation **Grid Search** de **Fmask_cnn** en fonction du *nombre d'époques* et *loss* sur les données d'apprentissage et celles de validation

8/ Représenter graphiquement la matrice de confusion associée aux données de test selon **Fmask_cnn**. Déduire l'accuracy obtenu

Bon Travail ♣